ASSETBRIDGE | TEAM WORKFLOW GUIDE | GIT & GITHUB

Team Git Collaboration & Workflow Guide

How to Work in Your Personal Branch, Sync with dev, and Push Cleanly

Repository: https://github.com/BohaniLucas/AssetBridge.git

1. Team Branch Structure & Roles

Our project utilizes a 3-tier structure to isolate in-progress work and protect the demo codebase:

BRANCH	PURPOSE & ROLE	ASSIGNED TO	DIRECT PUSH?
`main`	Production Release & Final Hackathon Submission	**Team Lead**	**NEVER (Protected)**
`dev`	Shared Integration / Staging branch (all features land here)	**Entire Team**	**Via Merge / PR**
`fullstack/Lucas`	Lucas's dedicated feature & full-stack development branch	**Lucas**	**YES (Your branch)**
`fullstack/Rory`	Rory's dedicated feature & full-stack development branch	**Rory**	**YES (Your branch)**
`fullstack/Dokotela`	Dokotela's dedicated feature & full-stack branch	**Dokotela**	**YES (Your branch)**

2. First-Time Setup & Branch Onboarding

Run these steps once in PowerShell / Terminal when getting started on your laptop:

```
# 1. Clone the project (if setting up fresh)
git clone https://github.com/BohaniLucas/AssetBridge.git
cd AssetBridge

# 2. Fetch all branches created by team members on GitHub
git fetch origin

# 3. Switch to YOUR designated branch (choose your name):
git checkout fullstack/Rory       # If you are Rory
git checkout fullstack/Dokotela   # If you are Dokotela
git checkout fullstack/Lucas      # If you are Lucas

# 4. Confirm you are on your branch (asterisk * marks active branch)
git branch
```

KEY COLLABORATION PRINCIPLE: ALWAYS PULL BEFORE YOU WRITE

To prevent merge conflicts, make it an absolute habit to sync dev into your branch each morning.

When your feature is complete and tested, merge your branch into dev so your teammates get it.

DAILY WORKFLOW â¢ SYNCING, COMMITTING & MERGING TO DEV

3. Morning Routine: Syncing with dev (dev -> your branch)

Before writing any new code, pull teammate updates from dev into your branch:

```
# Step 1: Switch to dev and fetch newest commits from teammates
git checkout dev
git pull origin dev
# Step 2: Switch back to your personal branch
git checkout fullstack/<your-name>
# Step 3: Merge dev changes into your working branch
git merge dev
# Result: Your branch now has all teammate updates!
```

4. Writing Code & Committing in Your Branch

Work exclusively on your designated branch. Commit small, logical changes frequently:

```
# 1. Check modified files
git status
# 2. Stage your changes
git add .
# 3. Commit with an explanatory message
git commit -m "feat(tokens): integrated P2P token transfer modal"
# 4. Push your branch to GitHub backup
git push origin fullstack/<your-name>
```

5. Pushing Finished Work to dev (your branch -> dev)

When your feature is complete and verified, push it into dev using either method:

METHOD A: GITHUB PULL REQUEST (RECOMMENDED & SAFEST)

1. Push your branch: git push origin fullstack/<your-name>
2. Go to https://github.com/BohaniLucas/AssetBridge -> Click 'Compare & pull request'
3. **Target base branch must be 'dev' (NOT main!). Compare branch: 'fullstack/<name>'**
4. Click 'Create pull request' -> Click 'Merge pull request'. Dev is now updated!

```
# METHOD B: DIRECT TERMINAL MERGE (Fast command line workflow)
git push origin fullstack/<your-name>   # Ensure personal branch is pushed
git checkout dev                        # Switch to dev branch
git pull origin dev                     # Get latest changes from others
git merge fullstack/<your-name>         # Merge your finished work into dev
git push origin dev                     # Push updated dev to GitHub
git checkout fullstack/<your-name>      # Return to your branch for next task
```

CONFLICT RESOLUTION, GOLDEN RULES & QUICK CHEAT SHEET

6. How to Resolve Merge Conflicts (No-Panic Guide)

A conflict only happens when you and a teammate edited the exact same line in the same file.
Git pauses and asks you which version to keep. Here is how to resolve it cleanly in VS Code:

STEP-BY-STEP CONFLICT RESOLUTION WORKFLOW

1. Open VS Code: Conflicting files are highlighted in purple with an exclamation mark (!).
2. Click on the file: Above the highlighted code, VS Code displays 3 clickable helper buttons:
   - **â¢ Accept Current Change: Keeps your version of the code.**
   - **â¢ Accept Incoming Change: Keeps your teammate's version from dev.**
   - **â¢ Accept Both Changes: Keeps both blocks if both are needed.**

`3. Save file -> git add . -> git commit -m "fix: resolved merge conflict with dev" -> Push!`

7. AssetBridge Team Golden Rules

RULE 1: NEVER PUSH DIRECTLY TO 'main'	Only the team lead merges 'dev' into 'main' after end-to-end verification.
RULE 2: NEVER COMMIT SECRETS OR '.env' FILES	All API keys and secrets stay in local .env (strictly git-ignored).
RULE 3: SYNC WITH 'dev' DAILY	Frequent small merges prevent large merge headaches. Sync morning & evening.
RULE 4: TEST BEFORE PUSHING TO 'dev'	Run npm test in Public/App and Backend before merging into dev.

8. One-Page Quick Command Cheat Sheet

GOAL	COMMAND SEQUENCE
See branch & pending files	`git status`
Download new branches from GitHub	`git fetch origin`
Switch to your personal branch	`git checkout fullstack/<your-name>`
Update your branch with team changes	`git checkout dev && git pull && git checkout fullstack/<name> && git merge dev`
Save & commit your work	`git add . && git commit -m "description of change"`
Backup your work to GitHub	`git push origin fullstack/<your-name>`
Merge your work to dev (Terminal)	`git checkout dev && git pull && git merge fullstack/<name> && git push origin dev`
Discard local mistakes on a file	`git restore <file-name>`